

Real-world (persist and read back real data) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper `sqlite3`), **Advanced** (the mistakes that only show up with real, repeated traffic). Read Basic first even if you already know SQL — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

SQLite stores everything in one file. Python's built-in `sqlite3` module lets you open that file, run SQL commands against it, and get results back — no separate database server to install or run.

Things to use:

- `import sqlite3`
- `conn = sqlite3.connect("runs.db")` — opens (or creates) the file.
- `conn.execute("CREATE TABLE ...")` — makes a table, once, if it doesn't already exist.
- `conn.execute("INSERT INTO ...")` then `conn.commit()` — saves a new row; `commit()` is required or nothing is actually written.
- `conn.execute("SELECT * FROM ...").fetchall()` — reads rows back.

Intermediate Version

A SQLite connection doesn't save anything to disk the instant you `execute()` an `INSERT` — it stages the change, and `commit()` is what actually writes it. Forgetting `commit()` is a real, common bug: your code runs with no errors, but the data silently isn't there the next time you look.

`CREATE TABLE IF NOT EXISTS` matters specifically because your FastAPI app will likely re-run this statement every time it starts up — without `IF NOT EXISTS`, the second startup would crash trying to create a table that already exists.

The exact pieces:

- `conn = sqlite3.connect("runs.db", check_same_thread=False)` — SQLite connections are tied to the thread that created them by default; FastAPI can call your route from a different thread than the one that opened the connection, so this flag is often needed (more on the *real* fix for this in Advanced).
- `"CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT, input TEXT, output TEXT, created_at TEXT)"` — `AUTOINCREMENT` gives each row a unique, ever-growing `id` for free.
- **Always use `?` placeholders, never f-strings, for values going into SQL:**
`conn.execute("INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)", (input_text, output_text, timestamp))` — this is what prevents SQL injection; string-

formatting user input directly into a SQL string is a real, serious vulnerability, not just a style preference.

- `datetime.now(timezone.utc).isoformat()` for a real, sortable timestamp string.

Advanced Version

Two problems only show up once real, repeated HTTP traffic hits this: connections, and reading rows back as usable JSON.

Connection lifetime: opening a brand-new `sqlite3.connect()` on every single request works, but it's wasteful, and under concurrent requests SQLite can throw `database is locked` errors if two writes overlap badly. The real pattern: open one connection (or a small pool) when the app starts, reuse it across requests, and understand that SQLite allows many readers at once but only one writer at a time — which is fine for this exercise's traffic, but is exactly the ceiling that "move to Postgres later" (mentioned in this document's Core Concepts) is for.

Reading rows back as JSON: by default, `fetchall()` gives you plain tuples — `(1, "hi", "You said: hi", "2026-01-01T00:00:00")` — with no field names. Returning that directly from a FastAPI route either fails or produces JSON no client can use sensibly. The fix: `conn.row_factory = sqlite3.Row`, which makes each row behave like a dict (`row["input"]`, or `dict(row)` to convert it fully) instead of a bare tuple.

Pieces:

- `conn.row_factory = sqlite3.Row` — set once, right after connecting.
- `[dict(row) for row in conn.execute("SELECT * FROM runs").fetchall()]` — a list of real dicts, safe to return straight from a route.
- `conn = sqlite3.connect("runs.db", check_same_thread=False)` opened once at module level (or via FastAPI's startup event / a dependency), not inside every route function.

Difference between Basic, Intermediate, and Advanced: Basic names the tools for the tidy, single-request case. Intermediate explains *why* each piece matters — `commit()`, `IF NOT EXISTS`, `?` placeholders instead of string formatting — for code that will actually be restarted and called with real input. Advanced covers what only breaks under real, repeated traffic: connection reuse instead of one-per-request, SQLite's single-writer limit, and turning raw tuples into real, JSON-usable dicts — the exact gap between "works once, in a script" and "backs a running API."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
connect to runs.db
create table runs (id, input, output, created_at) if it doesn't exist

route POST /chat:
    get a reply the same way as chat_route
    insert (input, output, now) into runs, then commit
    return the reply

route GET /runs:
    select * from runs
    return the rows
```

Here's almost the whole thing:

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT, input TEXT, output TEXT, created_at TEXT)")

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute("INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
    conn.commit()
    return {"reply": reply}

@app.get("/runs")
```

```
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return rows
```

Expected problem you'll hit running this as-is: `GET /runs` returns something like `[[1, "hi", "You said: hi", "2026-01-01T00:00:00"]]` — a list of plain lists with no field names. That's the exact gap Intermediate fixes.

Intermediate Version

```
same setup, plus:
    conn.row_factory = sqlite3.Row    # rows behave like dicts now

route GET /runs:
    select * from runs
    convert each row to a real dict before returning
```

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT, input TEXT, output TEXT, created_at TEXT)")

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute("INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
    conn.commit()
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return [dict(row) for row in rows]
```

Expected output (`curl localhost:8000/runs` after one `POST /chat`):

```
[{"id":1,"input":"hi","output":"You said: hi","created_at":"2026-01-01T00:00:00.000000"}]
```

Advanced Version

same setup, plus response models for real validated output:

```
define RunRecord(BaseModel): id, input, output, created_at

route GET /runs, response_model=list[RunRecord]:
    select * from runs
    return them (FastAPI + Pydantic build the RunRecord objects for you from the

route GET /runs/{run_id}:
    select the one row matching run_id
    if it doesn't exist: raise a 404
    otherwise return it
```

Here's most of it — fill in the single-row lookup yourself:

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT, input TEXT, output TEXT, created_at TEXT)")

class ChatRequest(BaseModel):
    message: str

class RunRecord(BaseModel):
    id: int
    input: str
    output: str
    created_at: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
```

```

conn.execute("INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
conn.commit()
return {"reply": reply}

@app.get("/runs", response_model=list[RunRecord])
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return [dict(row) for row in rows]

@app.get("/runs/{run_id}", response_model=RunRecord)
def get_run(run_id: int):
    # your turn: SELECT * FROM runs WHERE id = ?, using (run_id,) as the params
    # if fetchone() gives back None, raise HTTPException(404, detail="run not found")
    ...

```

Test the 404 path on purpose: request an `id` you know doesn't exist and confirm you get a clean 404, not a 500 or a crash.

Difference between Basic, Intermediate, and Advanced: Basic saves and reads rows, but hands back raw, unlabeled tuples. Intermediate fixes that with `row_factory` and `dict(row)`, so `GET /runs` returns real, usable JSON. Advanced adds a response model that validates the shape of every returned row, and a single-row lookup by ID that has to handle "doesn't exist" cleanly — the exact `GET /runs/{id}` shape the Build Task's database layer needs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)